Android Setup Guide (Windows)

Expo CRUD Starter: from a fresh Windows PC to the app running on an Android emulator

Written for people with no previous React Native or Android experience. Follow the parts in order. Each grey box is a command you can copy into **PowerShell**.

How to open PowerShell: press the **Windows key**, type *PowerShell*, press Enter. When a step says **"as Administrator"**, right-click *Windows PowerShell* and choose **Run as administrator**.

Platform: Windows 10 (64-bit, version 1809 or newer) or Windows 11. **Time:** about 1–2 hours the first time, mostly downloads.

Contents

1. Why we need Android Studio (native modules)
2. What you need before starting
3. Prepare Windows (virtualization, long paths)
4. System setup: install the tools
5. Android Studio: first-time setup wizard
6. Android Studio: SDK Manager (what to tick)
7. Environment variables (JAVA_HOME, ANDROID_HOME)
8. Create an emulator (virtual phone)
9. ADB: connecting emulators and real phones
10. Run the application
11. Daily workflow
12. Troubleshooting
13. Quick checklist

1. Why we need Android Studio (native modules)

Many Expo tutorials say "install the **Expo Go** app on your phone and scan a QR code". **That does not work for this project.**

What is a native module?

A React Native app has two layers:

- **JavaScript/TypeScript layer:** our screens, logic and styling (everything in `src\`).
- **Native layer:** real Android (Kotlin/Java/C++) code compiled into the app. Libraries that need phone features or very high speed ship their own native code. These are called **native modules**.

This project uses several libraries with native code, for example:

Library	What it does in our app
`react-native-mmkv` + `react-native-nitro-modules`	Fast, encrypted local storage (login token, theme, language)
`react-native-reanimated` + `react-native-worklets`	Smooth animations
`react-native-keyboard-controller`	Keyboard handling on forms
`react-native-gesture-handler`, `@gorhom/bottom-sheet`	Swipes and bottom sheets
`react-native-restart`	Restarts the app after changing language

`expo-dev-client`	Turns our app into a "development build" with a developer menu

Why Expo Go is not enough

Expo Go is a ready-made app from the Play Store. It only contains a **fixed set** of native modules chosen by Expo. Libraries like **MMKV / Nitro Modules** and **react-native-restart** are not inside it. If you open our project in Expo Go, it crashes with a "native module not found / not supported in Expo Go" type of error.

The solution: a development build

We compile **our own** version of the app on our computer. It contains every native module we use. This is called a **development build**. Compiling an Android app needs:

- **JDK 17:** Java, used by the Android build tool (Gradle)
- **Android SDK:** Android libraries and build tools
- **An emulator or a real phone** to install the app on

Android Studio is the easiest way to install the Android SDK and the emulator. We don't write code in Android Studio (we use VS Code or Cursor). We only use it for setup and to start the emulator.

```
pnpm android
    │
    ├─ 1. Expo generates the  android\  folder (native project)
    ├─ 2. Gradle + JDK 17 + Android SDK compile it  →  app-debug.apk   (slow, first time 10–20 min)
    ├─ 3. ADB installs the APK on the emulator / phone
    └─ 4. Metro bundler serves our JavaScript  →  app opens          (fast, every time)

Later, JS/TS changes  →  only step 4 (instant Fast Refresh)
New native library or app.config.ts change  →  run  pnpm android  again
```

2. What you need before starting

Item	Requirement
Operating system	Windows 10 64-bit (version 1809+) or Windows 11
Memory (RAM)	8 GB minimum, **16 GB recommended** (the emulator uses a lot)
Free disk space	**At least 30 GB** on drive C: (Android SDK + emulator image + project). An SSD is strongly recommended.
CPU	Intel or AMD 64-bit with **virtualization** support (almost every modern PC)
Internet	Stable connection. Several GB of downloads the first time.
Access	Administrator rights on the PC and access to the GitHub repository
Expo account	**Not needed** to run the app locally

> **iPhone / iOS:** iOS apps can only be built on a Mac. On Windows you develop and test on **Android**.

3. Prepare Windows (virtualization, long paths)

3.1 Check that virtualization is enabled

The emulator needs CPU virtualization, or it will be extremely slow or won't start.

1. Press `Ctrl + Shift + Esc` to open **Task Manager**.
2. Open the **Performance** tab → **CPU**.
3. Bottom right, look for **Virtualization: Enabled**.

If it says **Disabled**, restart the PC, enter the BIOS/UEFI (usually `F2`, `F10`, `Del` or `Esc` during startup), and enable **Intel VT-x / Intel Virtualization Technology** or **AMD-V / SVM Mode**. Save and restart. If you're not sure how, ask IT.

3.2 Turn on Windows Hypervisor Platform

Open PowerShell **as Administrator** and run:

```
Enable-WindowsOptionalFeature -Online -FeatureName HypervisorPlatform -All
```

Type `Y` to restart when asked.

Or by clicking: Windows key → type *Turn Windows features on or off* → tick **Windows Hypervisor Platform** → OK → restart.

3.3 Enable long file paths (important!)

Android builds create very deep folders. By default Windows limits paths to 260 characters, and the build fails with strange C++/CMake errors. Run in PowerShell **as Administrator**:

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force
```

> **Also:** keep the project in a **short folder** such as `C:\dev\expo-crud-starter`. Avoid paths like `C:\Users\Your Name\OneDrive\Desktop\Projects\…`. Long paths, **spaces** and **OneDrive** folders cause build errors.

4. System setup: install the tools

Windows 10/11 includes **winget**, the built-in Windows package installer. We use it instead of downloading each installer by hand. Open a **normal** PowerShell window (not Administrator) and check that it works:

```
winget --version
```

If `winget` is not found, install or update **App Installer** from the Microsoft Store, then reopen PowerShell.

4.1 Install Node.js, Git, Java 17 and Android Studio

```
winget install -e --id OpenJS.NodeJS.LTS
winget install -e --id Git.Git
winget install -e --id Microsoft.OpenJDK.17
winget install -e --id Google.AndroidStudio
```

Accept any prompts (type `Y`, or click **Yes** on Windows permission popups).

- **Node.js LTS:** runs JavaScript tools (version 20 or newer is required)
- **Git:** downloads the code from GitHub
- **Microsoft OpenJDK 17:** Java Development Kit 17, needed by Android builds. **Use 17, not newer.**
- **Android Studio:** installs the Android SDK manager and emulator

> **Close PowerShell and open a new window** after these installs. Otherwise the new commands are not found.

4.2 Allow PowerShell to run npm/pnpm scripts

By default PowerShell blocks script files, which stops `pnpm` from running. Run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Type `Y` and press Enter if asked.

4.3 Install pnpm (our package manager)

```
npm install -g pnpm
```

> **Important:** this project **only** works with `pnpm`. `npm install` and `yarn` are blocked on purpose.

4.4 Allow Git to handle long paths

```
git config --global core.longpaths true
```

4.5 Check the versions

```
node -v
pnpm -v
java -version
git --version
```

Expected: Node **v20** or higher, pnpm **10.x**, Java **"17.x"**. If `java -version` shows a different version or is not found, finish Part 7 first, then check again.

Watchman is not needed on Windows.

5. Android Studio: first-time setup wizard

Open the Start menu → **Android Studio**.

Screen you see	What to select
Import Android Studio Settings	**Do not import settings** → OK
Help improve Android Studio (data sharing)	Your choice: **Don't send** is fine
Welcome	**Next**
Install Type	**Standard** → Next
Select UI Theme	Any (Dark / Light) → Next
Verify Settings	Check that it lists **Android SDK**, **Android SDK Platform**, **Android Emulator** and (on Intel/AMD PCs) **Android Emulator hypervisor driver** → Next
License Agreement	Click **each** item on the left (e.g. *android-sdk-license*, *android-sdk-preview-license*) and choose **Accept** for every one → Finish
Downloading Components	Wait. This can take 10–30 minutes. Click **Yes** on any Windows permission popup (for the hypervisor driver). Then click **Finish**.

> The SDK is installed in `C:\Users\<your-name>\AppData\Local\Android\Sdk` (in PowerShell: `$env:LOCALAPPDATA\Android\Sdk`). We'll use this path in Part 7.

> If the wizard does not appear, or you skipped it by mistake, open Android Studio and go to **More Actions** → **SDK Manager** and follow Part 6.

6. Android Studio: SDK Manager (what to tick)

On the Android Studio welcome screen, click **More Actions** → **SDK Manager**. If a project is open, use **File** → **Settings** → **Languages & Frameworks** → **Android SDK** instead.

Tab 1: SDK Platforms

Tick **Show Package Details** (bottom right), then expand **Android 16.0 ("Baklava") – API Level 36** and tick:

- ☑ **Android SDK Platform 36**
- ☑ **Google Play Intel x86_64 Atom System Image** (normal Intel/AMD PCs)
  Only on ARM laptops (e.g. Snapdragon / Copilot+ PCs): **Google Play ARM 64 v8a System Image**

Expo SDK 54 builds against API 36. If you need to test older Android versions, you can install additional system images later.

Tab 2: SDK Tools

Tick **Show Package Details** and make sure these are ticked:

- ☑ **Android SDK Build-Tools** → expand and tick **36.0.0**
- ☑ **Android SDK Command-line Tools (latest)**
- ☑ **Android Emulator**

- ☑ **Android SDK Platform-Tools** (this contains `adb`)
- ☑ **Android Emulator hypervisor driver (installer)** (Intel/AMD PCs)
- ☑ **Google USB Driver** (only needed to use a real **Pixel** phone over USB)
- □ NDK (Side by side) and CMake are **optional**. The build downloads the right versions automatically when needed.

Click **Apply** → **OK**, accept the licenses, and wait for the download to finish.

7. Environment variables (JAVA_HOME, ANDROID_HOME)

Windows needs to know where Java and the Android SDK are. Copy this whole block into a **normal** PowerShell window and run it **once**:

```
$jdk = (Get-ChildItem "C:\Program Files\Microsoft" -Directory -Filter "jdk-17*" | Select-Object
-First 1).FullName
$sdk = "$env:LOCALAPPDATA\Android\Sdk"
[Environment]::SetEnvironmentVariable("JAVA_HOME", $jdk, "User")
[Environment]::SetEnvironmentVariable("ANDROID_HOME", $sdk, "User")
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$userPath;$sdk\platform-tools;$sdk\emulator", "Us
er")
Write-Host "JAVA_HOME = $jdk"
Write-Host "ANDROID_HOME = $sdk"
```

Check that both printed paths are real folders. `JAVA_HOME` should look like `C:\Program Files\Microsoft\jdk-17.0.x.x-hotspot`.

Now **close every PowerShell / VS Code window** and open a new PowerShell. Then check:

```
echo $env:JAVA_HOME
echo $env:ANDROID_HOME
java -version
adb --version
emulator -version
```

All five commands should print something. None of them should say "*is not recognized as the name of a cmdlet*".

Prefer clicking? Set them by hand

1. Windows key → type *environment variables* → open **Edit environment variables for your account**.
2. Under **User variables** click **New…**:
   - Name `ANDROID_HOME` → Value `C:\Users\<your-name>\AppData\Local\Android\Sdk`
   - Name `JAVA_HOME` → Value `C:\Program Files\Microsoft\jdk-17.0.x.x-hotspot` (check the exact folder name in File Explorer)
3. Select **Path** → **Edit…** → **New**, and add both:
   - `%ANDROID_HOME%\platform-tools`
   - `%ANDROID_HOME%\emulator`
4. Click **OK** on every window, then reopen PowerShell.

8. Create an emulator (virtual phone)

1. Open Android Studio. On the welcome screen click **More Actions** → **Virtual Device Manager**.
2. Click **+ (Create Virtual Device)**.
3. **Select Hardware:** category **Phone** → choose **Pixel 8** (any recent Pixel is fine). Prefer devices with the **Play Store** icon → **Next**.
4. **System Image:** choose **API 36** (*Baklava* / *Android 16*), **Google Play**, ABI **x86_64**. On ARM laptops, choose **arm64-v8a**. If a download icon ⬇ appears next to it, click it and wait. Then click **Next**.
5. **Verify Configuration:**
   - AVD Name: keep the default, e.g. `Pixel_8` (no spaces is easiest)
   - Startup orientation: Portrait
   - Click **Show Advanced Settings** (in newer versions this is the *Additional settings* tab):
     - Graphics: **Hardware** (or *Automatic*)
     - RAM: **4096 MB** if your PC has 16 GB, otherwise **2048 MB**
     - Internal Storage: **8 GB** or more (our debug app is large)

   Click **Finish**.
6. In the Device Manager list, click ▶ **(Play)** next to your device. A phone window opens and boots Android. The first boot can take 1–3 minutes.

Start the emulator from PowerShell (optional, faster)

```
emulator -list-avds          # shows your emulator names, e.g. Pixel_8
emulator -avd Pixel_8        # starts it (keep this window open)
```

If the emulator acts strangely, use a cold boot: `emulator -avd Pixel_8 -no-snapshot-load`

> If you get "*x86_64 emulation currently requires hardware acceleration*", go back to Part 3.1 and 3.2. Virtualization must be enabled and Windows Hypervisor Platform must be on.

9. ADB: connecting emulators and real phones

ADB (Android Debug Bridge) is the tool that lets your PC talk to an Android device. It installs the app and forwards network ports. It comes with *Android SDK Platform-Tools*.

9.1 Check connected devices

With the emulator running:

```
adb devices
```

Expected output:

```
List of devices attached
emulator-5554   device
```

Status shown	Meaning	Fix
`device`	Connected and ready ✅	Nothing to fix
(empty list)	No device found	Start the emulator, or plug in your phone and enable USB debugging (9.3)
`offline`	Device is still booting or stuck	Wait 30 seconds, or run the restart commands in 9.2
`unauthorized`	Phone hasn't trusted this PC	Unlock the phone and tap **Allow** on the "Allow USB debugging?" popup

9.2 Restart ADB (fixes most connection problems)

```
adb kill-server
adb start-server
adb devices
```

9.3 Use a real Android phone instead of an emulator (USB)

1. On the phone, open **Settings** → **About phone** and tap **Build number 7 times**. You'll see "*You are now a developer*". On Samsung it's under *Software information*; on Xiaomi, tap *MIUI/OS version*.
2. Go to **Settings** → **System** → **Developer options** and turn on **USB debugging**.
3. Connect the phone to the PC with a **data** USB cable. Some cables only charge.
4. **Windows USB driver:** Pixel phones use the **Google USB Driver** (Part 6). Samsung, Xiaomi, OnePlus and other brands need the manufacturer's USB driver (search "*<brand> USB driver Windows*").
5. On the phone, tap **Allow** (tick *Always allow from this computer*).
6. Run `adb devices`. Your phone's serial number should appear with status `device`.
7. Let the phone reach the local mock API through the USB cable:

   ```
   adb reverse tcp:3001 tcp:3001     # mock API
   adb reverse tcp:8081 tcp:8081     # Metro bundler (Expo usually does this automatically)
   ```

   With `adb reverse`, the phone can use `http://localhost:3001/` in `.env`. Run these again each time you reconnect the cable.

9.4 Wireless debugging (Android 11+, optional)

Phone and PC must be on the same Wi-Fi. On the phone, open **Developer options** → **Wireless debugging** → **Pair device with pairing code**. Then on the PC:

```
adb pair <ip>:<pairing-port>      # enter the 6-digit code shown on the phone
adb connect <ip>:<port>           # the IP:port shown on the Wireless debugging screen
adb devices
```

9.5 Several devices connected?

If `adb devices` lists more than one device, `pnpm android` asks which one to use. You can also pick one directly:

```
pnpm android --device
```

10. Run the application

10.1 Get the code (into a short folder)

```
mkdir C:\dev -Force
cd C:\dev
git clone https://github.com/saiphoenix/expo-crud-starter.git
cd expo-crud-starter
```

If GitHub asks you to log in, a browser window opens. Sign in. If the repository is private, ask the owner to add you as a collaborator first.

10.2 Install dependencies (the first command after cloning)

```
pnpm install
```

This downloads all libraries into `node_modules\` and takes a few minutes.

10.3 Set the API address in `.env`

The project includes a `.env` file that points to a local fake backend (the **mock API**). An Android emulator can't reach your PC at `localhost`. It uses the special address **10.0.2.2** instead.

Where the app runs	`EXPO_PUBLIC_API_URL` in `.env`
Android **emulator**	`http://10.0.2.2:3001/`
Real phone over USB with `adb reverse` (9.3)	`http://localhost:3001/`
Real phone over Wi-Fi	`http://<your-PC-IP>:3001/` (find the *IPv4 Address* with `ipconfig`)
Remote demo API (no local backend)	`https://dummyjson.com/`

For the emulator, run:

```
(Get-Content .env) -replace 'localhost:3001', '10.0.2.2:3001' | Set-Content .env
Get-Content .env
```

You should see `EXPO_PUBLIC_API_URL=http://10.0.2.2:3001/`. You can also open it with `notepad .env` and edit it by hand.

10.4 Start the mock API (PowerShell window 1, keep it open)

```
pnpm mock:api
```

Expected: 🚀 `Mock API on http://localhost:3001`. Every product you create, edit or delete is saved in `mock-api\db.json`.

> **Windows Firewall popup:** if Windows asks whether to allow **Node.js** on the network, tick **Private networks** and click **Allow**.

10.5 Start the emulator

Start it from Android Studio (Part 8, step 6) or with `emulator -avd Pixel_8`. Confirm with `adb devices`.

10.6 Build and run the app (PowerShell window 2)

Open a **second** PowerShell window (or a new tab in Windows Terminal with `Ctrl + Shift + T`), then run:

```
cd C:\dev\expo-crud-starter
pnpm android
```

What happens:

1. Expo creates the `android\` folder. This is generated code; never edit it by hand.
2. Gradle compiles the native app. **The first time takes 10–20 minutes** on Windows and downloads a lot. Later builds are much faster.
3. The app is installed on the emulator and opens automatically.
4. Metro (the JavaScript bundler) keeps running in this window. Leave it open.

10.7 Use the app

1. Onboarding screen → tap **Let's Get Started**
2. Login → this is a demo login. Use **any email** and **any 6-character password**, e.g. `test@test.com` / `123456`
3. Tabs: **Feed** (template example), **Products** (our CRUD reference), **Style** (UI kit), **Settings**

> **Done!** Change any text in `src\` and save. The app updates instantly (Fast Refresh).

11. Daily workflow (after the first build)

You do **not** need to run `pnpm android` every day. Use:

```
# Window 1 — backend
cd C:\dev\expo-crud-starter; pnpm mock:api

# Window 2 — start the emulator first, then:
cd C:\dev\expo-crud-starter; pnpm start -c
#   then press  a  to open the app on Android
```

When…	Run
You only changed JS/TS code	Nothing. Fast Refresh updates the app.
You changed `.env`	Stop Metro (Ctrl + C) → `pnpm start -c`
You pulled new code with new packages	`pnpm install`
A **native** library was added, or `app.config.ts` changed	`pnpm install` → `pnpm android` (rebuild)
Open the developer menu in the app	Press `m` in the Metro window, or `Ctrl + M` in the emulator
Reload the app	Press `r` in the Metro window

12. Troubleshooting

Problem / error message	Fix
"*… is not recognized as the name of a cmdlet*" (pnpm, adb, java, emulator)	Close **all** PowerShell / VS Code windows and reopen. If it still fails, redo Part 7.
"*pnpm.ps1 cannot be loaded because running scripts is disabled*"	Run Part 4.2: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`
App crashes with a native module error, e.g. "*not supported in Expo Go*"	You opened the project in **Expo Go**. Uninstall Expo Go from the emulator, then run `pnpm android` to install our development build (Part 1).
`SDK location not found. Define ANDROID_HOME…`	Redo Part 7. Check that `echo $env:ANDROID_HOME` prints a folder that exists.
`Unsupported class file major version` / Java version errors	The wrong Java version is in use. Make sure `java -version` shows **17** and `JAVA_HOME` points to `jdk-17…` (Part 7).
CMake / ninja errors, "*Filename longer than 260 characters*", "*No such file or directory*" inside `.cxx`	Path too long. Do Part 3.3 (LongPathsEnabled), move the project to `C:\dev\expo-crud-starter`, delete the `android` folder and rebuild.
Emulator: "*requires hardware acceleration*" or extremely slow	Enable virtualization in the BIOS and Windows Hypervisor Platform (Parts 3.1 and 3.2). Use an **x86_64** image. Set Graphics to **Hardware**.
`No Android connected device found`	Start the emulator and check `adb devices` (Part 9). If needed, run `adb kill-server; adb start-server`.
Real phone not shown in `adb devices`	Use a data cable, enable USB debugging, install the phone's USB driver (9.3), tap **Allow** on the phone.
`INSTALL_FAILED_INSUFFICIENT_STORAGE`	Device Manager → ⋮ next to the emulator → **Wipe Data**, or increase Internal Storage (Part 8).
Products tab shows **Network Error**	Check that `pnpm mock:api` is running, that `.env` uses `10.0.2.2` (emulator), that Node.js is allowed through the Windows Firewall, then restart with `pnpm start -c`.
Red screen: "*Unable to load script*"	Metro isn't running. Run `pnpm start` and press `a`. On a real phone, run `adb reverse tcp:8081 tcp:8081`.
`Port 8081 already in use`	`npx kill-port 8081` then try again

Gradle build fails with strange errors	Clean and rebuild:
	`cd android; .\gradlew clean; cd ..`
	`pnpm android`
	If it still fails: `Remove-Item -Recurse -Force android` then `pnpm android`. The folder is regenerated.

"*EPERM: operation not permitted*" / file locked errors	Close VS Code and other terminals, stop Metro, and try again. If the project is inside OneDrive, move it to `C:\dev`.
Builds are very slow	Normal the first time. Optionally add `C:\dev` and `%USERPROFILE%\.gradle` to the Windows Security exclusions, but only if your IT policy allows it.
Old screens or weird behaviour after a change	Stop Metro → `pnpm start -c` (clears the cache)

`pnpm install` fails with **401 Unauthorized**	Your PC points to a private package registry. Run: `npm config set registry https://registry.npmjs.org/` and try again.
"*Use pnpm instead*" / `only-allow` error	You ran `npm install` or `yarn`. Use `pnpm install`.
Still stuck?	Run `pnpm doctor` and share the output and the full error with the team.

13. Quick checklist

✓	Step	Check with
☐	Virtualization enabled, Windows Hypervisor Platform on, long paths enabled	Task Manager → CPU
☐	Node, Git, JDK 17, Android Studio, pnpm installed	`node -v; pnpm -v; java -version`
☐	Android Studio setup wizard finished (Standard)	`dir $env:LOCALAPPDATA\Android\Sdk`
☐	SDK Platform 36, Build-Tools 36, Platform-Tools, Emulator, Command-line Tools	SDK Manager
☐	JAVA_HOME, ANDROID_HOME and Path set	`echo $env:ANDROID_HOME`
☐	Emulator created and booted	`adb devices` → `device`
☐	Repo cloned into `C:\dev` and `pnpm install` done	`node_modules\` exists
☐	`.env` → `http://10.0.2.2:3001/`	`Get-Content .env`
☐	Mock API running	`pnpm mock:api`
☐	App built and opened	`pnpm android`
☐	Logged in and Products tab loads	In the app

More details in the repository: `docs/01-getting-started.md`, `docs/12-environments-and-builds.md`, `docs/14-debugging-and-common-mistakes.md`, `docs/16-mock-api.md`.